

Guía del Backend: Pygestor

Este documento explica en detalle cómo funciona el backend del proyecto Pygestor (Gestor de Tareas), para qué sirve cada componente y cómo puedes modificarlo.

Estructura del Backend

El backend de Pygestor está construido con **Flask** (un micro-framework web de Python) y **SQLAlchemy** (un ORM para interactuar con la base de datos).

La estructura de archivos es la siguiente:

- `run.py` : El punto de entrada de la aplicación.
- `app/__init__.py` : Inicializa la aplicación y la base de datos.
- `app/models.py` : Define la estructura de la base de datos.
- `app/routes.py` : Define las rutas (URLs) y la lógica de la aplicación.
- `app/pygestor.db` : El archivo de base de datos SQLite (se genera automáticamente).

1. `run.py` - El Punto de Entrada

Para qué sirve: Es el archivo que ejecutas para iniciar el servidor web. Llama a la función que crea la aplicación y la pone a escuchar peticiones. **Cómo se usa:** Se ejecuta en la terminal con `python run.py`. **Cómo editarlo:** Rara vez necesitas editar este archivo. Si quisieras cambiar el puerto en el que corre la app (por defecto 5000), podrías cambiar `app.run(debug=True)` por `app.run(debug=True, port=8080)`.

2. `app/__init__.py` - Inicialización

Para qué sirve: Configura la aplicación Flask, establece la conexión con la base de datos SQLite y registra las "rutas" (Blueprint) para que la app sepa cómo responder a las URLs. **Cómo se usa:** Es importado automáticamente por `run.py`. **Cómo editarlo:**

- Si quieres cambiar el nombre de la base de datos, busca la línea `app.config['SQLALCHEMY_DATABASE_URI']` y cambia `pygestor.db` por otro nombre.
- Si en el futuro quieres añadir extensiones como un sistema de login (Flask-Login), lo inicializarías aquí.

3. `app/models.py` - Base de Datos

Para qué sirve: Define cómo se guardan los datos. En lugar de escribir SQL puro, escribes clases de Python.

Cómo se usa: Define una clase `Task` con columnas (id, title, description, status, created_at). **Cómo editarlo:**

- Si quieres añadir una fecha de vencimiento a las tareas, añadirías una línea dentro de la clase `Task` :
`due_date = db.Column(db.DateTime, nullable=True)`.
- Si quieres crear una nueva tabla (por ejemplo, Usuarios), crearías una nueva clase `class`
`User(db.Model):` debajo de `Task`.

4. `app/routes.py` - Lógica y Rutas

Para qué sirve: Es el cerebro de la aplicación. Define qué pasa cuando un usuario visita una URL específica (ej.

`/`, `/add`, `/update/1`). **Cómo se usa:** Utiliza decoradores como `@bp.route('/')` para conectar una URL con una función de Python. **Cómo editarlo:**

- **Para añadir una nueva página:** Creas una nueva función. Por ejemplo:

```
@bp.route('/about')
def about():
    return "Página acerca de Pygestor"
```

- **Para modificar cómo se añaden las tareas:** Editas la función `add()` . Por ejemplo, si añadiste `due_date` en `models.py` , aquí tendrías que capturar ese dato de `request.form.get('due_date')` y pasarlo al crear la nueva `Task` .

Resumen

Si quieres añadir una nueva funcionalidad, el flujo normal es:

1. **models.py:** (Opcional) Modificas o añades la tabla en la base de datos.
2. **routes.py:** Creas la URL y la lógica en Python.
3. **Frontend:** (HTML/CSS) Creas el diseño visual para interactuar con esa ruta.